

computed and returned as the result.

```
(asoc-do ((key value) alist [result]
body ...)) ;; Syntax
```

```
(let ((sum 0)
      (alphabets '((b . 2) (a . 1) (e . 5) (d . 4) (c . 3))))
  (asoc-do ((key value) alphabets sum)
    (when (symbolp key)
      (setf sum (+ sum value))))
  sum)
15
```

The anaphoric variant of the *asoc-do* function is the *asoc-do* construct and is illustrated below:

```
(asoc--do (alist &rest body)) ;; Syntax
```

```
(let ((alphabets '((b . 2) (a . 1) (e . 5) (d . 4) (c . 3))))
  (asoc--do alphabets

    (when (symbolp key)
      (setf result (+ (or result 0)
value))))))
15
```

Mapping functions

The *asoc-map* function takes two arguments — a function and an *alist*. It applies the function to each and every element in the *alist*. In the following example, the key of every element in the *alist* is checked to see if it is a symbol and then its value is returned.

```
(asoc-map function alist) ;; Syntax
```

```
(asoc-map (lambda (k v) (when (symbolp k) v))
  '((b . 2) (a . 1) (e . 5) (d . 4) (c . 3)))
(2 1 5 4 3)
```

The anaphoric variant of the *asoc-map* function is the *asoc-map* function and is demonstrated below with an example:

```
(asoc--map form alist) ;; Syntax
```

```
(asoc--map (format "%s=%d;" key value)
  '((b . 2) (a . 1) (e . 5) (d . 4) (c . 3)))
("b=2;" "a=1;" "e=5;" "d=4;" "c=3;")
```

You can make a transformation on the keys on the given *alist* using the *asoc-map-keys* function. The keys are converted into strings in the following example:

```
(asoc-map-keys function alist) ;;
Syntax
```

```
(asoc-map-keys #'symbol-name '((b . 2)
(a . 1) (e . 5) (d . 4) (c . 3)))
(("b" . 2) ("a" . 1) ("e" . 5) ("d" . 4) ("c" . 3))
```

If you want to make transformations on the values, you can use the '*asoc-map-values*' function as shown below:

```
(asoc-map-values function alist) ;;
Syntax
```

```
(let ((alphabets '((b . 2) (a . 1) (e . 5) (d . 4) (c . 3))))
  (asoc-map-values #'list alphabets))
((b 2) (a 1) (e 5) (d 4) (c 3))
```

Folds

The fold operation is a reduction on an association list. The *asoc-fold* function takes three arguments — a reduction function, an *alist* and an initial value (accumulator). In the following example, the values are added to the accumulator and the sum of the values is returned.

```
(asoc-fold function alist init) ;;
Syntax
```

```
(let ((alphabets '((b . 2) (a . 1) (e . 5) (d . 4) (c . 3))))
  (asoc-fold (lambda (k v acc) (+ v
acc))
    alphabets 0))
15
```

The anaphoric variant of the *asoc-fold* function is the *asoc-fold* function. Its syntax and usage are demonstrated with the following example:

```
(asoc--fold form alist init) ;; Syntax
```

```
(let ((alphabets '((b . 2) (a . 1) (e . 5) (d . 4) (c . 3))))
  (asoc--fold (+ value acc) alphabets 0))
15
```

You can merge multiple *alists* using the '*asoc-merge-values*' function. For example:

```
(asoc-merge-values alists) ;; Syntax
```

```
(let ( (vowels '((a . 1) (e . 5) (i . 9)))
      (consonants '((b . 2) (c . 3) (d . 4))))
  (asoc-merge-values vowels consonants))
((a 1) (e 5) (i 9) (b 2) (c 3) (d 4))
```

If you have duplicate keys, then you can use the *asoc-merge-values-no-dups* function, which will merge multiple unique values for each key to a single *alist*. An example is shown below:

```
(asoc-merge-values-no-dups alists) ;;
Syntax
```

```
(let ( (vowels '((a . 1) (e . 5) (i . 9)
(a . 1)))
      (consonants '((b . 2) (c . 3) (d . 4) (d . 4))))
  (asoc-merge-values-no-dups vowels
consonants))
((a 1) (e 5) (i 9) (b 2) (c 3) (d 4))
```

Do explore the README at <https://github.com/troyp/asoc.el> to learn more about the association list functions given above and their usage. 

 By: Shakthi Kannan

The author is a free software enthusiast who blogs at shaktimaan.com.